KDU 동아리 프로젝트 보고서

경동대학교 재학생을 위한 동아리 탐색·가입·개설 웹사이트

학번: _____ 이름: _____

기준일: 2026년 10월 14일 · 기준 소스: 854fef220164

자료 확인 및 작성 범위

현재 프로젝트 소스와 Git 이력, 최초 업로드 ZIP, 제공된 2회차 미션 설명문 및 이번 실행 로그를 분석했습니다. 교수 체크리스트는 과제 안내문 7장의 원문을 사용했습니다. 이번 제출의 AI 초안은 사용자가 지정한 UI 최초 적용 커밋 2fdda11입니다. UI 이전의 최초 AI 프롬프트와 학생 본인이 직접 작성한 코드 범위는 확인 필요입니다. 학번·이름은 작성란으로 남겼습니다.

이 보고서의 실행 화면은 별도 제출 소스를 임시 DB와 로컬 서버로 실행해 2026년 10월 14일에 캡처했습니다. 실제 학교의 동아리 데이터나 학적 정보가 아니라 화면 검증용 가상 데이터입니다. 모바일, 학교 SSO, 실제 학교 승인, 이메일 발송은 구현 범위에 포함하지 않습니다.

1. 프로젝트 개요

KDU 동아리는 여러 동아리를 한곳에서 찾아보고 가입이나 개설을 신청할 수 있는 웹사이트입니다. 학생, 동아리 개설자, 교수 계정이 각각 필요한 일을 할 수 있도록 화면과 처리 권한을 구분했습니다. 웹브라우저에서 화면을 사용하고 서버는 Python Flask로 동작합니다.

2. 개발 목적

사용자는 동아리 활동이 충분히 활성화되지 않고 정보를 찾기 어렵다는 문제를 제시했습니다. 경동대학교 재학생이 관심 분야의 동아리를 쉽게 찾도록 검색과 필터를 제공하고, 신청 결과를 내 활동과 알림에서 확인하도록 했습니다. 대화에서 교수의 답변은 “검색필터나 알림 기능을 넣는다면 오케이”로 전달되었습니다. 실제 카카오워크 승인 원본과 시작 시점은 확인 필요입니다.

3. 개발 환경 및 사용 기술

구분	사용 기술	역할
언어	Python 3.12 / HTML·CSS·JavaScript	서버 처리와 화면 구성
서버	Flask 3.1.2 / Werkzeug 3.1.9	URL 연결, 요청·세션·응답, 비밀번호 해시
화면	Jinja2 3.1.6	DB에서 읽은 데이터를 HTML에 표시
DB	SQLite (Python sqlite3)	사용자·동아리·신청·공지·질문·알림 저장
파일	Pillow 11.3.0 / pypdf 6.1.1	이미지 내용 검사 / 등록 PDF 파싱
검증	unittest / Playwright·Chromium	서버 기능과 실제 브라우저 동작 확인

실행 의존성은 requirements.txt에 고정했습니다. 문서 제작에 사용한 python-docx, python-pptx, LibreOffice 및 Graphviz는 제출물을 만드는 도구로, 웹사이트 실행에는 필요하지 않습니다. Windows 실행 명령은 안내했지만 사용자 PC에서의 실제 실행은 확인 필요입니다.

4. 주요 기능

기능	동작
동아리 탐색	이름·소개·캠퍼스 검색, 분야 필터, 모집 중 필터, 찜
가입 신청	지원동기·참여 가능 여부·목표·가치관 제출, 대기 신청 취소
개설 신청	이름·분야·소개로 신청, 상세 계획·PDF 선택, 교수 검토·보완 후 재신청
내 활동	신청 현황과 개설 신청 두 탭. 가입 동아리와 찜은 신청 현황에 통합
동아리 운영	활동 정보, 모집 회차·기간·정원·조기 마감, 부원 심사
활동 자료	공지 작성·수정과 첨부, 일정 작성·수정·삭제
Q&A	익명 공개 질문과 질문자·개설자만 볼 수 있는 비밀 질문
알림	신청 결과 알림, 전체/안읽음 필터, 읽음 및 결과 이동

공지 첨부는 PDF/HWP/DOCX/JPG/PNG를 허용하고 파일별 10MiB 및 공지당 최대 3개로 제한합니다. 등록 PDF는 선택이고 파일명은 자유입니다. 첨부한다면 10MiB 이하의 읽을 수 있는 암호화되지 않은 PDF여야 합니다. 공식 최소 인원·캠퍼스·필수 서류 확인 제한은 최신 요청에 따라 해제했습니다.

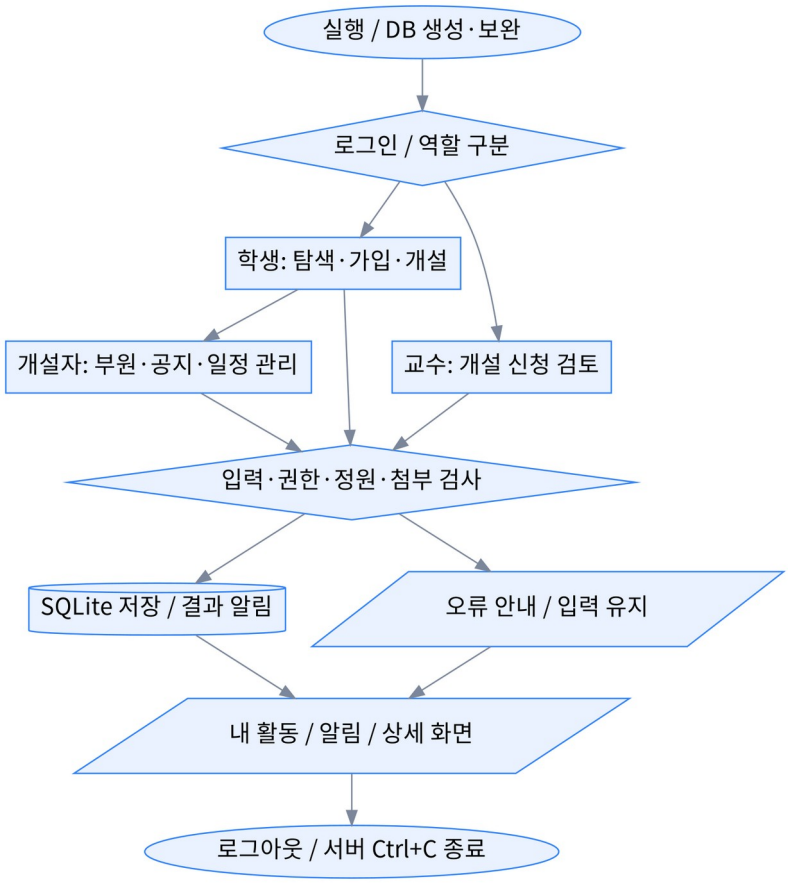
5. 시스템 구성 및 프로그램 구조

브라우저가 URL이나 폼을 요청하면 Flask의 해당 함수가 입력과 권한을 확인합니다. SQLite에서 조회하거나 변경한 뒤 Jinja2 템플릿으로 화면을 만들고 응답합니다. 파일은 디스크에 저장하고 DB에는 파일 이름과 저장 이름, 소유자, 연결된 공지 등의 정보를 기록합니다.

```
kdu_club_project/
  app.py                  # 서버와 기능 처리
  schema.sql              # 새 DB 초기 구조
  seed_demo.py            # 가상 실행 데이터 생성
  requirements.txt        # 실행 의존성
  templates/              # HTML 화면
  static/                 # CSS·JS·SVG·홍보 이미지
  tests/                  # 임시 DB 자동 검증
  qa/                     # 실제 브라우저 검증
```

핵심 테이블은 users, clubs, club_members, recruitment_rounds, club_posts, attachments, club_schedules, club_qna, club_favorites, notifications, registration_documents입니다. 사용자는 여러 동아리에 가입할 수 있고 각 가입 신청은 모집 회차와 연결됩니다. 기존 전체 부원 수와 현재 신규 모집 승인 수를 구분합니다.

6. 프로그램 동작 흐름



전체 동작 개요. 조건·오류·파일 저장을 포함한 상세 순서도는 01_flowchart/flowchart.png와 PDF에 있습니다.

python app.py 실행 시 시연 DB가 없으면 먼저 생성합니다. DB가 있으면 삭제하지 않고 빠진 구조만 추가합니다. 로그인 성공 시 학생은 탐색 화면, 교수는 개설 검토 화면으로 이동합니다. 요청마다 CSRF와 역할·소유자 조건을 검사합니다. 저장 성공 시 다음 화면으로 이동하고 입력 오류는 작성 내용을 유지하면서 안내합니다. 로그아웃은 세션만 비우며 서버는 계속 실행됩니다. 터미널에서 Ctrl+C를 누르면 서버가 종료됩니다.

7. 주요 코드 및 기능 설명

7.1 로그인

아이디 조회는 바인딩을 사용합니다. 저장된 비밀번호 해시와 입력값을 비교하고, 성공하면 세션을 초기화한 뒤 역할을 저장합니다.

근거: app.py 269행의 login()

```
def login():
    if request.method == 'POST':
        user = row('SELECT * FROM users WHERE username=?', (request.form.get('username',
'').strip(),))
        if user and check_password_hash(user['password'], request.form.get('password', '')):
            session.clear()
            session.update(user_id=user['id'], username=user['username'], name=user['name'],
role=user['role'])
            return redirect(url_for('professor_dashboard' if user['role'] == 'professor' else
'index'))
        return render_template('login.html', error='아이디 또는 비밀번호를 다시 확인해 주세요.',
values=request.form)
    return render_template('login.html', values={})
```

7.2 검색과 필터

사용자의 검색어를 SQL 문자열 안에 직접 넣지 않고 물음표와 인자 목록을 사용합니다. 분야 별칭도 고정한 조건으로 처리합니다.

근거: app.py 307행의 index()

```
    keyword = request.args.get('keyword', '').strip()
    category = request.args.get('category', 'all')
    query = "SELECT * FROM clubs WHERE status='approved'"
    params = []
    if keyword:
        query += ' AND (name LIKE ? OR description LIKE ? OR campus LIKE ?)'
        params.extend([f'%{keyword}%'] * 3)
    if category != 'all':
        aliases={'예술':['예술','공연'],'사회·봉사':['사회·봉사','봉사']}
        selected=aliases.get(category,[category])
        query += ' AND category IN ('+','.join('?' for _ in selected)+')'
        params.extend(selected)
```

7.3 부원 승인과 알림

개설자 권한을 확인하고 쓰기 잠금을 확보한 다음 정원을 다시 셉니다. 대기 상태에서만 결과를 저장하고 해당 신청자의 ID에 알림을 기록합니다.

근거: app.py 651행의 manage_member()

```
        mem=row('SELECT * FROM club_members WHERE id=?',(mem['id'],))
        if mem['status']=='pending':
            if action=='approve' and mem['recruitment_id']:
                r=row('SELECT * FROM recruitment_rounds WHERE id=? AND club_id=?',
(mem['recruitment_id'],club['id']))
                count=row("SELECT COUNT(*) n FROM club_members WHERE recruitment_id=? AND
status='approved'",(mem['recruitment_id'],))['n']
                if not r or count>=r['capacity']:
                    raise ValueError('이번 모집 정원을 초과해 승인할 수 없습니다.')
            new_status='approved' if action=='approve' else 'rejected'
            db.execute("UPDATE club_members SET status=?, rejection_reason=?,reviewed_at=? WHERE
id=? AND status='pending'",
                       (new_status,rejection,now().isoformat(timespec='minutes'),mem['id']))
            notify(mem['user_id'],f"{club['name']} 가입 신청이 {'승인' if action=='approve' else '반려'}
되었어요.",'membership',club['id'],mem['id'])
        db.commit()
```

7.4 파일 검증

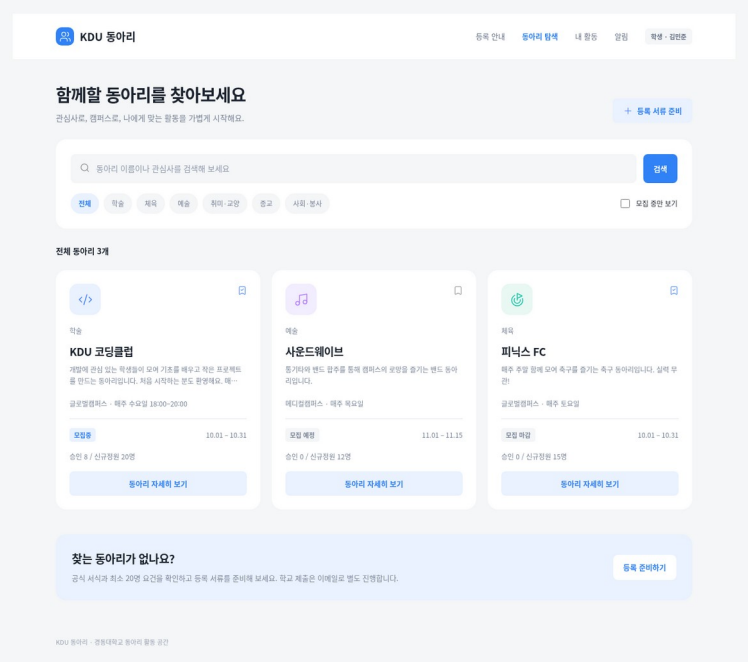
공지 첨부의 파일 크기와 실제 형식을 확인합니다. 이미지에는 Pillow 검사를 적용하고, 저장 이름은 별도 UUID를 사용해 사용자가 보낸 파일 경로를 그대로 쓰지 않습니다.

근거: app.py 744행의 validated_upload()

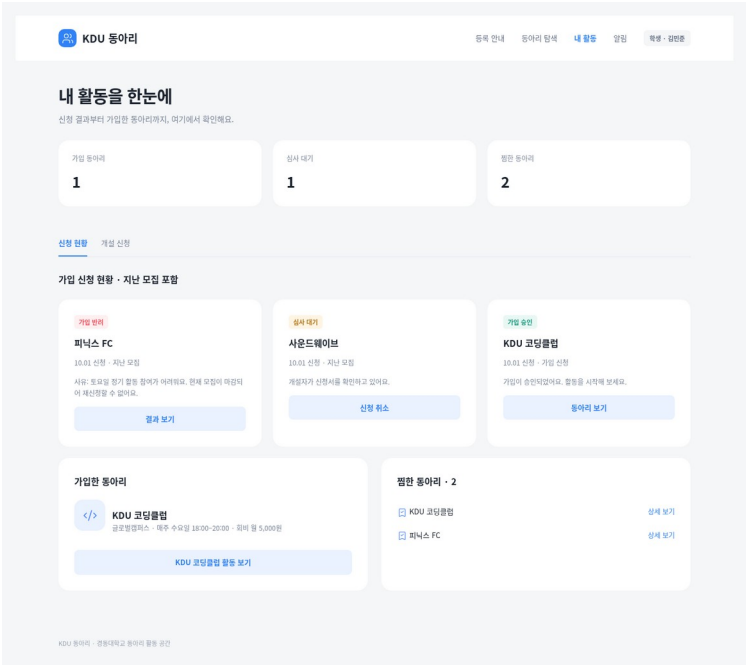
```
    """용량·실제 형식을 검사합니다. 파일 이름은 표시용으로만 사용합니다."""
    data=file.read(10*1024*1024+1)
    if len(data)>10*1024*1024:
        raise ValueError('파일은 각각 10MB 이하여야 합니다. 작성 내용은 유지됩니다.')
    if not data:
        raise ValueError('빈 파일은 첨부할 수 없습니다.')
    ext=Path(file.filename or '').suffix.lower()
    if ext in {'.jpg','.jpeg','.png'}:
        try:
            with Image.open(io.BytesIO(data)) as image:
                if image.format not in ({'PNG'} if ext=='.png' else {'JPEG'}):
```

공지 PDF 검사는 헤더 확인이며, 등록 PDF는 pypdf로 파싱합니다. HWP와 DOCX 검사는 대표 형식·구조 검사입니다. 악성코드 검사나 문서 내용의 진위 확인까지 수행한 것은 아닙니다.

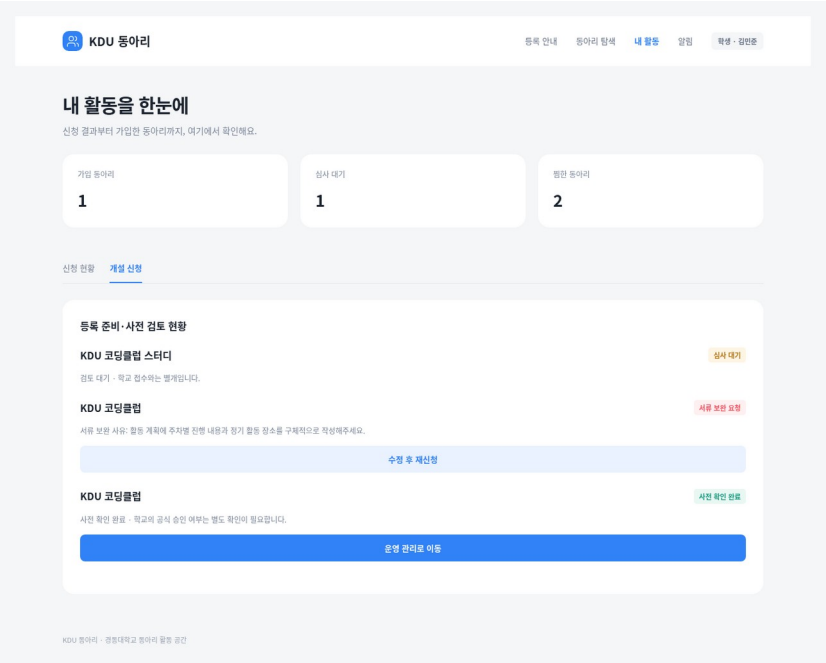
8. 실제 실행 화면



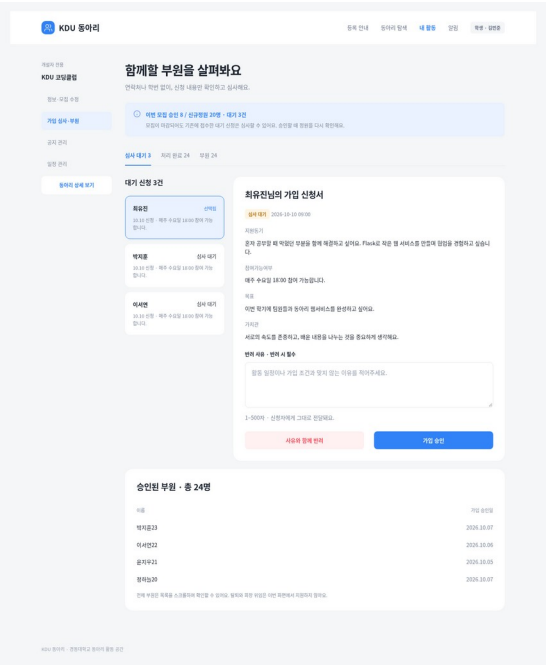
동아리 탐색: 검색·분야·모집 필터와 카드



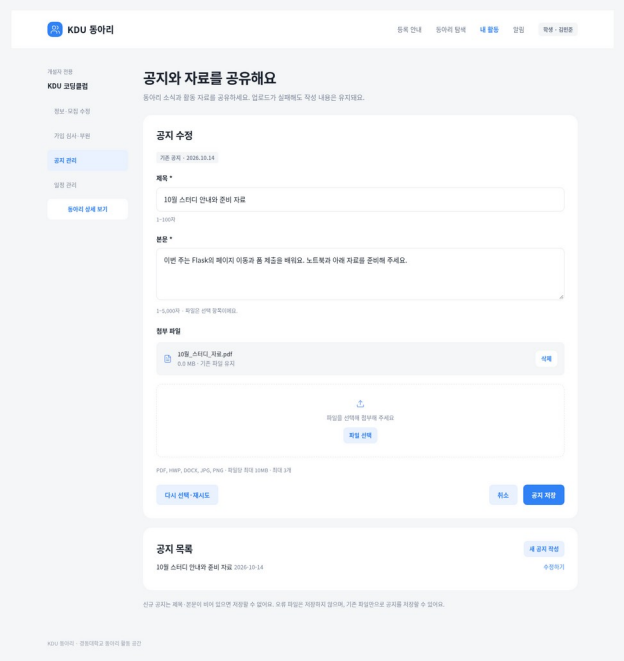
내 활동: 신청 결과·가입 동아리·찜 목록



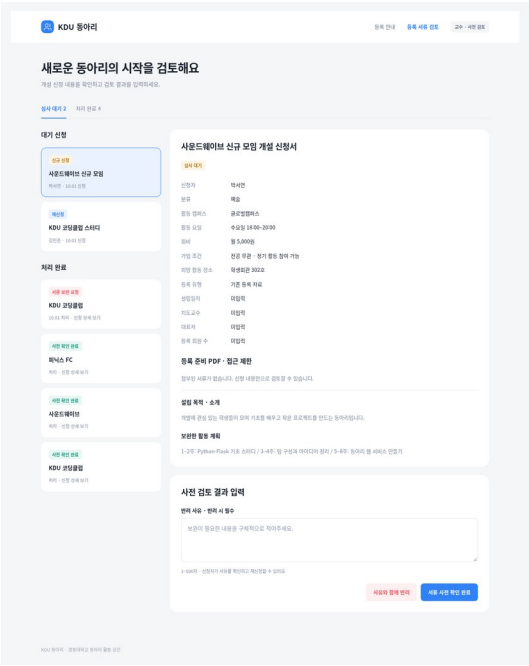
내 활동: 개설 신청 탭



개설자: 신청 상세 확인과 가입 승인·반려



공지 편집: 본문과 기존 첨부 유지



교수: 개설 신청과 검토 결과 입력

02_report/screenshots에는 전체 크기의 실제 캡처를 함께 보관했습니다. 미구현 화면을 임의로 제작하지 않았습니다.

최초 Figma와 같은 크기의 픽셀 비교는 확인되지 않았으며 현재 검증은 화면 표시와 기능 동작 기준입니다.

9. 테스트 및 검증 결과

항목	이번 실행 결과	근거
제출 소스 자동 테스트	42개 통과	evidence/source_tests.log
주석 복사본 자동 테스트	42개 통과	evidence/commented_tests.log
주석 동작 동일성	Python AST 일치	evidence/ comment_ast_equivalence.json
브라우저	전체 검사 통과·기록된 콘솔 오류 0	evidence/browser_checks.log / browser-results.json
개인 Windows PC	확인 필요	사용자 PC 설치·실행 결과가 아직 제공되지 않음
제출용 UI 초안	Git 2fdda11 확보	실제 커밋 소스·파일 해시 보존

테스트는 임시 DB로 실행하여 기존 사용자 데이터를 변경하지 않았습니다. 검사 중 일부 파일 응답에서 ResourceWarning이 기록될 수 있으며 테스트 실패로 집계되지는 않았습니다. 통과 결과를 모든 운영 환경에 대한 보장으로 해석하지 않습니다.

10. 발생한 문제와 해결 방법

문제	확인된 해결	근거
로그인 이후 500 화면	시스템 시간대 DB 의존성 제거, 빈 모집 날짜·잘못된 정원 처리, 초기화 SQL UTF-8 지정	60734a8 및 관련 회귀 테스트. 사용자 PC의 정확한 Traceback은 미확인
중복 심사와 잘못된 알림 수신	대기 상태 변경 때만 알림 생성, 사용자·소유자 권한 검사	c694b39 / 2fdda11
첨부 실패 시 입력 유실 우려	업로드 단계 분리, 오류 시 공지 본문과 기존 첨부 유지	2fdda11, 브라우저 업로드 실패 검사
실행 방법 불편	python app.py로 시연 DB 자동 생성 및 기존 DB 보존	cf7f92e
Windows pip 명령 미인식	py -m pip 및 가상환경의 python.exe로 설치하도록 안내	대화 기록. 사용자 PC 해결 여부는 확인 필요

11. AI 활용 과정

사용자는 서비스 아이디어와 역할·기능을 설명하고 기존 코드 기반의 보완을 요청했습니다. 이후 Figma 데스크톱 캡처를 제공하고 UI·기능 구현, 직접 실행, 로그인 오류 수정, 공식 서식 반영 후 제한 해제, 탭 통합과 제출용 문구 정리를 요청했습니다. 대화에서 AI가 실제 코드 수정·테스트·커밋을 수행한 것이 확인됩니다. 학생 본인이 직접 코딩한 범위를 동일하게 취급하지 않았습니다.

이번 제출은 사용자가 지정한 UI 최초 적용 버전 2fdda11을 AI 초안으로 사용합니다. 실제 Git 소스를 추출했으며 현재 코드로 과거 버전을 재구성하지 않았습니다. 그 이전 업로드 ZIP을 이번 초안이라고 표시하지 않습니다. 05_ai_initial_draft와 06_ai_revision_verification에 UI 초안과 최종 코드의 출처·차이를 정리했습니다. 학생이 직접 수행할 수 있는 수정·검증 후보는 별도 문서에 제공하며 실제 수행 전에는 본인 작업으로 기록하지 않습니다.

12. 결과 및 느낀 점

결과: 동아리 탐색부터 가입·개설 신청과 검토, 운영 관리, 알림까지 하나의 흐름으로 연결했습니다. 필수 추가 기능 중 업로드·검색/필터·알림을 구현하고 과제 7장 조건을 점검했습니다. 학교 인증·운영 배포·모바일은 이번 범위에서 검증하지 않았습니다.

느낀 점 작성 초안 — 학생 본인의 검토 후 수정 필요: “이번 프로젝트를 통해 화면이 보이는 것과 기능이 정확하게 동작하는 것이 다르다는 점을 배울 수 있었습니다. 특히 신청 상태와 권한을 서버에서 확인하고 잘못된 파일이나 중복 요청을 검증하는 과정이 필요하다는 것을 이해했습니다. AI가 만든 코드도 직접 읽고 실행 결과를 확인해야 한다고 느꼈습니다.” 이는 실제 학생 경험을 확인한 진술이 아닌 발표·작성용 초안입니다.

13. 참고자료

교수 제공: 2회차_학생배포_미션설명문.pdf (7장 체크리스트·8장 제출물·9장 제출 방법). 원문은 evidence에 보존했습니다.

프로젝트 Git 저장소: https://github.com/jongjong123alt/testest, 분석 기준 854fef22016485e0ea1c797e288bae226a8268f5

UI 최초 적용 Git 2fdda11 및 이 대화의 요구·수정 요청. 업로드 ZIP은 개발 이전 자료로만 검토했습니다.

Flask 공식 문서: https://flask.palletsprojects.com/

Python sqlite3 문서: https://docs.python.org/3/library/sqlite3.html

Jinja 문서: https://jinja.palletsprojects.com/

Werkzeug 보안 도구: https://werkzeug.palletsprojects.com/

Pillow·pypdf 문서: https://pillow.readthedocs.io/ 및 https://pypdf.readthedocs.io/

Figma 캡처 제공 디자인: H60BlkYObkcv42S9hJcBdI. MCP 원본 접근·정확한 픽셀 비교는 미확인.